



Assignment

Title : Online Recipe Management System

Batch : 03.01

Group Number : 09

Declaration:

We hold a copy of this assignment that we can produce if the original is lost or damaged.

We hereby certify that no part of this assignment has been copied from any other group's work or from any other source. No part of this assignment has been written/produced for our group by another person except where such collaboration has been authorized by the subject lecturer/tutor concerned.

Group Members :

IT23183018

HIRUSHA D.G.A.D


.....
Signature

IT23191006

COORAY Y.H.


.....
Signature

IT23195202

HARSHANA L.L.E.


.....
Signature

IT23189294

GUNATHILAKA H.D.I.


.....
Signature

IT23190566

GANEGODA R.B.


.....
Signature

Submitted on : 04 / 05 / 2024

Contents

1. Hypothetical Scenario
2. 2. Requirement Analysis Document
 - 2.1. Overview
 - 2.2. Functional Requirements
 - 2.3. Non-functional Requirements
3. Data Requirements
4. ER Model
5. Relational Schema
6. SQL Commands
 - 6.1. Creating Tables
 - 6.2. Insert Values into the Tables
7. Special Performance
8. Special Security Requirements

1. Hypothetical Scenario

A recipe is a set of instructions or guidelines for preparing a particular dish or beverage. It typically includes a list of ingredients with specific quantities, as well as step-by-step directions for combining and cooking those ingredients to produce the intended outcome. Recipes can vary in complexity, from simple instructions for basic dishes to more elaborate preparations requiring specialized techniques and ingredients. They serve as a roadmap for cooks to follow in order to recreate a specific dish accurately.

An Online Recipe Management System is a digital platform designed to help users organize, store, search and share recipes. These systems typically offer a range of features to make managing recipes more convenient and efficient. Recipe storage, meal planning, cooking notes, organization, search functionality, sharing and collaboration are some common features .

A user can create and input recipes, import recipes, organize recipes, edit and modify, plan meals, discover new recipes and collaborate with others.

System admin has access to verify recipe content, manage user accounts, update recipe details, verify payments and calculate discounts . The manager can add payments, approve updated recipes, grant discounts , manage FAQ and generate reports.

The Online Recipe Management System will store all the essential data in the system database for future use. All details about recipes, users will be stored and displayed to make decisions.

02. Requirements Analysis Document

2.1 Overview

The database is structure of data that stores structured or meaningful and related data . Database is an essential component of a system. A database is used to store large volumes of data. It can create relationships among data. In databases, primary keys, and foreign keys support retrieval of data from several database table.

This Online Recipe Management System requires a database as there is a large volume of data to be stored. All the data related to users, Employees and recipes will be stored in the database. In here, online recipe management system uses a database to give users a better user experience. Also, the users can use their stored data to make decisions. The database system also provides high security for collecting data. So that unauthorized people cannot access.

2.2. Functional Requirements

1. A guest can browse through the recipe details without registering.
2. Register a guest to the system.
3. Registered user can view the recipes.
4. Registered user should be able to get recipe details.
5. Provide access to the registered user to add feedback.
6. Registered user should be able to make payments with cash, PayPal, AMEX, Master, or VISA.
7. Relevant language can be chosen .
8. Admin can add recipe content like photos, videos.
9. Accounts are managed by admin.
10. Admin should manage FAQs.
11. Staff details are managed by admin.
12. Admin send a verify confirmation message, validate payments.
13. Seasonal and package discounts are calculated by the admin.
14. Manager adds payments and update accounts.
15. Manager approves updated recipes.
16. Annual, monthly and weekly reports are generated by manager.
17. Manager grants discounts.

2.3. Non-Functional Requirements

01. Usability :

The system must be easy to use and understand. This can be viewed in many perspectives.

- Time it take to execute.
- Tasks users can complete without assistance.
- How simple and straightforward the user interface are?

02. Security :

The system must be secure from unauthorized access. This also provides anti-virus and anti-malware protection.

03. Performance :

The system must be able to handle the required number of users without any degradation in performance.

04. Scalability :

The system must be able to scale up or down as needed.

05. Availability :

The system must be available when needed. A good system provides service 24-7.

3. Data Requirements

- Registered_User
 - User ID (User_ID)
 - Username (Username)
 - First Name (First_Name)
 - Last Name (Last_Name)
 - Country (Country)
 - Gender (Gender)
 - Date Of Birth (Date_Of_Birth)
 - Age (Age)
 - Is Premium (Is_Premium)
 - Is Creator (Is_Creator)
 - Admin ID (Admin_ID)
 - Profile Image (Profile_Image)
- Premium_Membership
 - Package ID (Pkg_ID)
 - Package Name (Pkg_Name)
 - Duration (Duration)
 - Price (Price)
- Premium_User
 - User ID (User_ID)
 - Package ID (Pkg_ID)
 - Package Started Date (Pkg_Started_Date)
 - Package Due Date (Pkg_Due_Date)
- Creator
 - Creator ID (Creator_ID)
 - User ID (User_ID)
- Admin
 - Admin ID (Admin_ID)
 - First Name (First_Name)
 - Last Name (Last_Name)
 - Email (Email)
 - Contact number (Contact_No)
 - Gender (Gender)
 - Admin Password (Admin_Password)
 - Profile Image (Profile_Image)

- Manager
 - Manager ID (Manager_ID)
 - First Name (First_Name)
 - Last Name (Last_Name)
 - Email (Email)
 - Contact Number (Contact_No)
 - Gender (Gender)
 - Manager Password (Manager_Password)
 - Profile Image (Profile_Image)
- Report
 - Report ID (Report_ID)
 - Manager ID (Manager_ID)
 - Report Type (Report_Type)
 - Report Frequency (Report_Frequency)
 - Submitted Date (Submitted_Date)
 - Report File Path (Report_File_Path)
- Payment_Method_details
 - Method ID (Method_ID)
 - Method Name (Method_Name)
- Email
 - User ID (User_ID)
 - Email (Email)
- Contact
 - User ID (User_ID)
 - Contact Number (Contact_No)

- Recipe
 - Recipe ID (Recipe_ID)
 - Creator ID (Creator_ID)
 - Title (Title)
 - Method (Method)
 - Rating (Rating)
 - Category (Category)
 - Published_Date (Published_Date)
 - Recipe Image (Recipe_Image)

- Blog_Post
 - Post ID (Post_ID)
 - User ID (User_ID)
 - Title (Title)
 - Post Type (Post_Type)
 - Published Date (Published_Date)

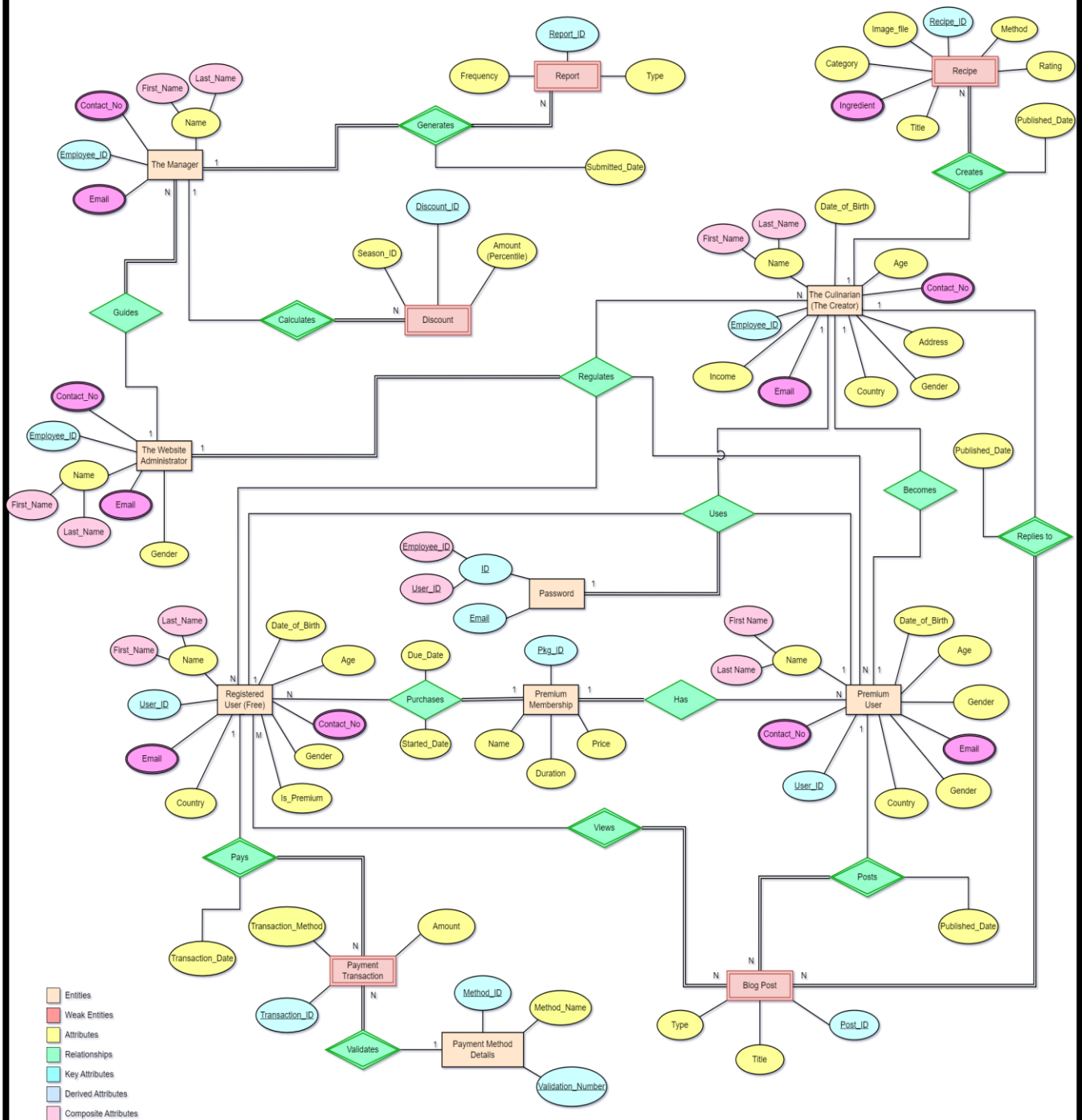
- Transaction
 - Transaction ID (Transaction_ID)
 - Card ID (Card_ID)
 - Method ID (Method_ID)
 - Transaction Date (Transaction_Date)
 - Card Type (Card_Type)
 - Amount (Amount)
 - Expiry Date (Exp_Date)

- Discount
 - Manager ID (Manager_ID)
 - Season ID (Season_ID)
 - Amount (Amount)

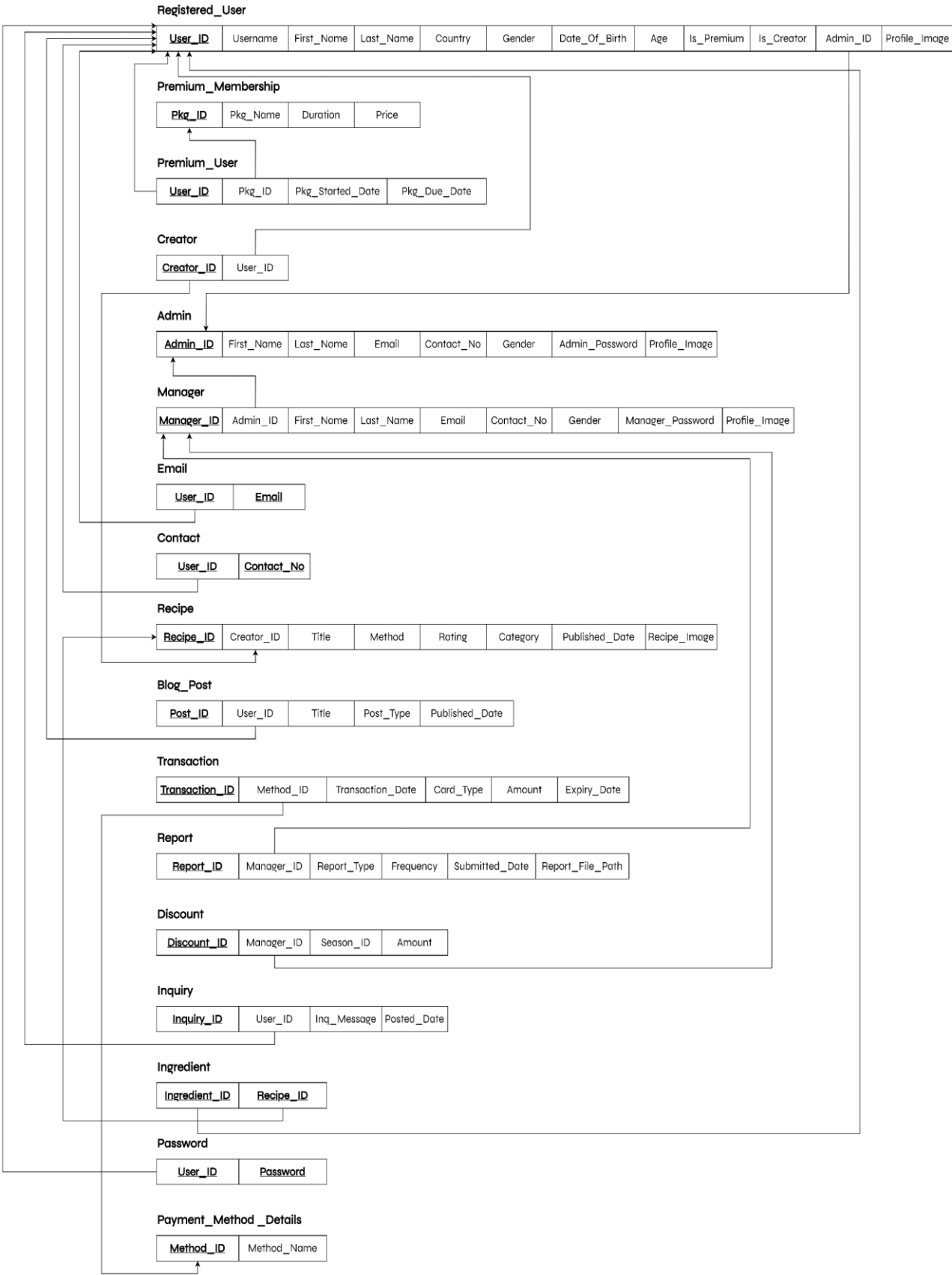
- Inquiry
 - Inquiry ID (Inquiry_ID)
 - User ID (User_ID)
 - Inquiry Message (Inq_Message)
 - Posted Date (Posted_Date)

- Password
User ID (User_ID)
User Password (User_ID)

4. The Entity Relationship Diagram



5. Relational Schema



6. SQL Commands

6.1. Creating Tables

-- Creating the Database

```
CREATE DATABASE recipeRealm;
```

-- Using the Database

```
USE recipeRealm;
```

-- Creating Registered User Table

```
CREATE TABLE Registered_User (  
    User_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    Username VARCHAR(50) NOT NULL,  
    First_Name VARCHAR(20) NOT NULL,  
    Last_Name VARCHAR(20) NOT NULL,  
    Country VARCHAR(20) NOT NULL,  
    Gender VARCHAR(10) NOT NULL,  
    Date_Of_Birth DATE NOT NULL,  
    Age INT NOT NULL,  
    Is_Premium BIT,  
    Is_Creator BIT,  
    Admin_ID INT NOT NULL,  
    Profile_Image VARCHAR(255) NOT NULL  
);
```

-- Creating Premium Membership Table

```
CREATE TABLE Premium_Membership (  
    Pkg_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    Pkg_Name VARCHAR(20) NOT NULL,  
    Duration VARCHAR(20) NOT NULL,  
    Price INT NOT NULL  
);
```

-- Creating Premium User Table

```
CREATE TABLE Premium_User (  
    User_ID INT PRIMARY KEY NOT NULL,  
    Pkg_ID INT NOT NULL,  
    Pkg_Started_Date TIMESTAMP NOT NULL,  
    Pkg_Due_Date DATE NOT NULL  
    CONSTRAINT premium_user_fk_user_id FOREIGN KEY(User_ID)  
REFERENCES Registered_User(User_ID) ON DELETE CASCADE,  
    CONSTRAINT premium_user_fk_pkg_id FOREIGN KEY(Pkg_ID)  
REFERENCES Premium_Membership(Pkg_ID) ON DELETE CASCADE,  
);
```

-- Creating Creator Table

```
CREATE TABLE Creator (  
    Creator_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    User_ID INT NOT NULL,  
    CONSTRAINT creator_fk_user_id FOREIGN KEY(User_ID)  
REFERENCES Registered_User(User_ID) ON DELETE CASCADE  
);
```

-- Creating Admins Table

```
CREATE TABLE Admins (  
    Admin_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    First_Name VARCHAR(30) NOT NULL,  
    Last_Name VARCHAR(30) NOT NULL,  
    Email VARCHAR(40) NOT NULL,  
    Contact_No VARCHAR(15) NOT NULL,  
    Gender VARCHAR(10) NOT NULL,  
    Admin_Password VARCHAR(50) NOT NULL,  
    Profile_Image VARCHAR(255) NOT NULL  
);
```

-- Creating Manager Table

```
CREATE TABLE Manager (  
    Manager_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    First_Name VARCHAR(30) NOT NULL,  
    Last_Name VARCHAR(30) NOT NULL,  
    Email VARCHAR(40) NOT NULL,  
    Admin_ID INT NOT NULL,  
    Contact_No VARCHAR(15) NOT NULL,  
    Gender VARCHAR(10) NOT NULL,  
    Manager_Password VARCHAR(50) NOT NULL,  
    Profile_Image VARCHAR(255) NOT NULL  
    CONSTRAINT manager_fk_admin_id FOREIGN KEY(Admin_ID)  
REFERENCES Admin(Admin_ID) ON DELETE CASCADE  
);
```

-- Creating Email Table

```
CREATE TABLE Email (  
    User_ID INT NOT NULL,  
    Email VARCHAR(40) NOT NULL,  
    CONSTRAINT email_table_pkey PRIMARY KEY(User_ID, Email),  
    CONSTRAINT email_fk_user_id FOREIGN KEY(User_ID)  
REFERENCES Registered_User(User_ID) ON DELETE CASCADE  
);
```

-- Creating Contact Table

```
CREATE TABLE Contact (  
    User_ID INT NOT NULL,  
    Contact_No VARCHAR(15) NOT NULL,  
    CONSTRAINT contact_table_pkey PRIMARY KEY(User_ID,  
Contact_No),  
    CONSTRAINT contact_fk_user_id FOREIGN KEY(User_ID)  
REFERENCES Registered_User(User_ID) ON DELETE CASCADE  
);
```

-- Creating Recipe Table

```
CREATE TABLE Recipe (  
    Recipe_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    Creator_ID INT NOT NULL,  
    Title VARCHAR(40) NOT NULL,  
    Method VARCHAR(500) NOT NULL,  
    Rating VARCHAR(1) NULL,  
    Category VARCHAR(30) NULL,  
    Published_Date TIMESTAMP NOT NULL,  
    Recipe_Image VARCHAR(255) NOT NULL,  
    CONSTRAINT recipe_fk_creator_id FOREIGN KEY(Creator_ID)  
REFERENCES Creator(Creator_ID) ON DELETE CASCADE  
);
```

-- Creating Blog Post Table

```
CREATE TABLE Blog_Post (  
    Post_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    User_ID INT NOT NULL,  
    Title VARCHAR(30) NOT NULL,  
    Post_Type VARCHAR(15) NOT NULL,  
    Published_Date TIMESTAMP NOT NULL,  
    CONSTRAINT blog_post_fk_user_id FOREIGN KEY(User_ID)  
REFERENCES Registered_User(User_ID) ON DELETE CASCADE  
);
```

-- Creating Payment Method Detail Table

```
CREATE TABLE Payment_Method_Detail (  
    Method_ID VARCHAR(5) PRIMARY KEY NOT NULL,  
    Method_Name VARCHAR(15) NOT NULL  
);
```

-- Creating Transaction Table

```
CREATE TABLE Transactions (  
    Transaction_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    Method_ID VARCHAR(5) NOT NULL,  
    Transaction_Date TIMESTAMP NOT NULL,  
    Card_Type VARCHAR(20) NOT NULL,  
    Amount VARCHAR(20) NOT NULL,  
    Exp_Date DATE NOT NULL,  
    CONSTRAINT transaction_fk_method_id FOREIGN  
KEY(Method_ID) REFERENCES Payment_Method_Detail(Method_ID)  
ON DELETE CASCADE  
);
```

-- Creating Report Table

```
CREATE TABLE Report (  
    Report_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    Manager_ID INT NOT NULL,  
    Report_Type VARCHAR(20) NOT NULL,  
    Report_Frequency VARCHAR(20) NOT NULL,  
    Submitted_Date TIMESTAMP NOT NULL,  
    Report_File_Path VARCHAR(255) NOT NULL,  
    CONSTRAINT report_fk_manager_id FOREIGN KEY(Manager_ID)  
REFERENCES Manager(Manager_ID) ON DELETE CASCADE  
);
```

-- Creating Discount Table

```
CREATE TABLE Discount (  
    Discount_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    Manager_ID INT NOT NULL,  
    Season_ID VARCHAR(15) NULL,  
    Amount INT NOT NULL,  
    CONSTRAINT report_fk_manager_id FOREIGN KEY(Manager_ID)  
REFERENCES Manager(Manager_ID) ON DELETE CASCADE  
);
```

-- Creating Inquiry Table

```
CREATE TABLE Inquiry (  
    Inquiry_ID INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    User_ID INT NOT NULL,  
    Inq_Message VARCHAR(500) NOT NULL,  
    Posted_Date TIMESTAMP NOT NULL,  
    CONSTRAINT inquiry_fk_user_id FOREIGN KEY(User_ID)  
REFERENCES Registered_User(User_ID) ON DELETE CASCADE  
);
```


-- Creating Ingredient Table

```
CREATE TABLE Ingredient (  
    Ingredient_ID INT IDENTITY(1,1) NOT NULL,  
    Recipe_ID INT NOT NULL,  
    CONSTRAINT ing_pkey PRIMARY KEY(Ingredient_ID,  
Recipe_ID),  
    CONSTRAINT ingredient_fk_recipe_id FOREIGN  
KEY(Recipe_ID) REFERENCES Recipe(Recipe_ID) ON DELETE  
CASCADE  
);
```

-- Creating Password Table

```
CREATE TABLE Passwords (  
    User_ID INT NOT NULL,  
    User_Password VARCHAR(50) NOT NULL,  
    CONSTRAINT pwd_pkey PRIMARY KEY(User_ID, User_Password),  
    CONSTRAINT inquiry_fk_user_id FOREIGN KEY(User_ID)  
REFERENCES Registered_User(User_ID) ON DELETE CASCADE  
);
```

6.2. Inserting values to the Tables

-- Inserting into Registered User

```
INSERT INTO Registered_User (Username, First_Name,  
Last_Name, Country, Gender, Date_Of_Birth, Age, Is_Premium,  
Is_Creator, Admin_ID, Profile_Image) VALUES ('dineth2002' ,  
'Dineth' , 'Hirusha' , 'Sri Lanka' , 'Male' , '2002-11-28' ,  
21 , 1 , 1 , 1 , '#');
```

```
INSERT INTO Registered_User (Username, First_Name,  
Last_Name, Country, Gender, Date_Of_Birth, Age, Is_Premium,  
Is_Creator, Admin_ID) VALUES ('eshan2002' , 'Eshan' ,  
'Harshana' , 'Sri Lanka' , 'Male' , '2002-08-18' , 21 , 1 ,  
1 , 1 , '#');
```

```
INSERT INTO Registered_User (Username, First_Name,  
Last_Name, Country, Gender, Date_Of_Birth, Age, Is_Premium,  
Is_Creator, Admin_ID) VALUES ('yashodha2003' , 'Yashodha' ,  
'Cooray' , 'Sri Lanka' , 'Female' , '2003-05-28' , 20 , 1 ,  
1 , 1 , '#');
```

```
INSERT INTO Registered_User (Username, First_Name,  
Last_Name, Country, Gender, Date_Of_Birth, Age, Is_Premium,  
Is_Creator, Admin_ID) VALUES ('ranidu2003' , 'Ranidu' ,  
'Bhasahana' , 'Sri Lanka' , 'Male' , '2003-11-21' , 20 , 1 ,  
0 , 1 , '#');
```

```
INSERT INTO Registered_User (Username, First_Name,  
Last_Name, Country, Gender, Date_Of_Birth, Age, Is_Premium,  
Is_Creator, Admin_ID) VALUES ('dinil2002' , 'Dinil' ,  
'Gunathilaka' , 'Sri Lanka' , 'Male' , '2002-05-07' , 21 , 1  
, 0 , 1 , '#');
```

--Inserting into Premium Membership

```
INSERT INTO Premium_Membership VALUES (1 , 'Bronze' , '3  
months' , 1000);
```

```
INSERT INTO Premium_Membership VALUES (2 , 'Silver' , '6  
months' , 1500);
```

```
INSERT INTO Premium_Membership VALUES (3 , 'Gold' , '12  
months' , 3000);
```

```

-- Inserting into Premium user
INSERT INTO Premium_User VALUES (1 , 'Gold' , '2024-05-02' ,
'2025-05-02');
INSERT INTO Premium_User VALUES (2 , 'Gold' , '2024-05-02' ,
'2025-05-02');
INSERT INTO Premium_User VALUES (3 , 'Gold' , '2024-05-02' ,
'2025-05-02');
INSERT INTO Premium_User VALUES (4 , 'Gold' , '2024-05-02' ,
'2025-05-02');
INSERT INTO Premium_User VALUES (5 , 'Gold' , '2024-05-02' ,
'2025-05-02');

-- Inserting into Admins
INSERT INTO Admins VALUES (1 , 'Jack' , 'Sparrow' ,
'admin@recipe.com' , '0112323231' , 'Male', 'lemmethrough' ,
'#');

-- Inserting into Manager
INSERT INTO Manager VALUES (1 , 'Davy' , 'Jones' ,
'manager@recipe.com' , '0112323231' , 'Male', 'lemmethrough'
, '#');

-- Inserting into Payment Method Details
INSERT INTO Payment_Method_Detail VALUES ('M1', 'Visa');
INSERT INTO Payment_Method_Detail VALUES ('M2', 'Master');
INSERT INTO Payment_Method_Detail VALUES ('M3', 'Paypal');

-- Inserting into Email
INSERT INTO Email VALUES (1 , 'dineth@recipe.com');
INSERT INTO Email VALUES (2 , 'eshan@recipe.com');
INSERT INTO Email VALUES (3 , 'yashodha@recipe.com');
INSERT INTO Email VALUES (4 , 'ranidu@recipe.com');
INSERT INTO Email VALUES (5 , 'dinil@recipe.com');

-- Inserting into Contact
INSERT INTO Contact VALUES (1 , '0772424521');
INSERT INTO Contact VALUES (2 , '0761541638');
INSERT INTO Contact VALUES (3 , '0706080877');
INSERT INTO Contact VALUES (4 , '0770377131');
INSERT INTO Contact VALUES (5 , '0702310863');

-- Inserting into Discount
INSERT INTO Discount VALUES (101, 'Summer2024', 10);
INSERT INTO Discount VALUES (102, 'Winter2024', 20);
INSERT INTO Discount VALUES (103, NULL, 15);
INSERT INTO Discount VALUES (104, 'Spring2025', 25);
INSERT INTO Discount VALUES (105, NULL, 30);

```

-- Inserting into Recipe

```
INSERT INTO Recipe VALUES (1, 'Spaghetti Carbonara', '1. Boil water and cook spaghetti according to package instructions. 2. In a skillet, cook diced pancetta until crispy. 3. In a bowl, whisk eggs, cheese, and pepper. 4. Drain spaghetti and add to skillet with pancetta. 5. Quickly add egg mixture and stir until creamy. Serve immediately.', '5', 'Pasta', '2023-05-12', 'https://example.com/spaghetti_carbonara.jpg');
```

```
INSERT INTO Recipe VALUES (2, 'Chicken Alfredo', '1. Cook pasta according to package instructions. 2. In a skillet, cook chicken until no longer pink. 3. Add garlic and cook until fragrant. 4. Stir in heavy cream and Parmesan cheese. 5. Add cooked pasta and toss until coated. Serve hot.', '4', 'Pasta', '2023-06-25', 'https://example.com/chicken_alfredo.jpg');
```

```
INSERT INTO Recipe VALUES (3, 'Vegetable Stir Fry', '1. Heat oil in a wok or skillet. 2. Add chopped vegetables and stir-fry until tender. 3. Mix soy sauce, ginger, and garlic in a bowl. 4. Pour sauce over vegetables and stir until coated. 5. Serve over rice or noodles.', '4', 'Vegetarian', '2023-07-10', 'https://example.com/vegetable_stir_fry.jpg');
```

```
INSERT INTO Recipe VALUES (4, 'Chocolate Chip Cookies', '1. Preheat oven to 350°F. 2. In a bowl, cream butter and sugars. 3. Beat in eggs and vanilla. 4. Mix in flour, baking soda, and salt. 5. Stir in chocolate chips. 6. Drop dough by rounded spoonfuls onto baking sheets. 7. Bake for 10-12 minutes or until golden brown.', '5', 'Dessert', '2023-08-20', 'https://example.com/chocolate_chip_cookies.jpg');
```

```
INSERT INTO Recipe VALUES (5, 'Caprese Salad', '1. Slice tomatoes and mozzarella cheese. 2. Arrange tomato and mozzarella slices on a plate. 3. Drizzle with olive oil and balsamic glaze. 4. Season with salt, pepper, and fresh basil leaves. 5. Serve immediately.', '4', 'Salad', '2023-09-05', 'https://example.com/caprese_salad.jpg');
```

```
-- Inserting into Passwords
```

```
INSERT INTO Passwords VALUES (1 , '123');  
INSERT INTO Passwords VALUES (2 , '234');  
INSERT INTO Passwords VALUES (3 , '345');  
INSERT INTO Passwords VALUES (4 , '456');  
INSERT INTO Passwords VALUES (5 , '567');
```

07. Special Performance

This online management system has some special performance.

- In this system the usability is very high. It is designed to be user-friendly, allowing users to interact with the system easily .
- User interface is made using a spectacular color palate to make it more user friendly .
- System's performance is very high ; hence the execution time is very low .
- The system is available to users 24/7 .
- System can handle large number of users without any degradation in performance.
- Mobile – friendly environment is created to give customer the best experience.
- System provides easy payment methods.

08. Special Security Requirements

Users tend to be more consider about the security of the system as they must enter their personal details, passwords and credit card information. As a result, ensuring the user's privacy is critical.

- System has a high level of security.
- System is secured from unauthorized access.
- System uses the data encryption method.
- Before granting the authorization to log into the system, system validates the user account.
- System is protected by powerful firewall.